AI AGENT × FUTURE NETWORK

以智驭网 | 2026 第一届 全国高校智能体网络创新挑战赛

面向全国高校学子的 Agent 原生网络创新实践平台。提交你的 Agent 镜像，完成赛题挑战， 用可量化的结果登上榜单。

查看打榜材料　浏览赛题

1 **准备 Agent 镜像**
基于赛事提供的开发镜像构建

2 **上传并检查**
完成结构与安全检查

3 **Docker 沙箱评测**
隔离运行并实时查看进度

4 **查看成绩**
分项得分与全国排名

赛事阶段

01
START
报名组队
完成账号注册与队伍信息确认，了解参赛协议和运行环境。

02
BUILD
方案开发
下载基础镜像，围绕赛题方向完成 Agent 设计与自测。

03

SUBMIT

镜像提交

导出 Docker 或 OCI 镜像归档，提交后完成平台结构与安全检查。

04

SCORE

评测打榜

在隔离沙箱中运行真实任务，查看报告、分项成绩与全国排名。

奖项设置

奖金共 10 万元

+ 奖金总额 10 万元
+ CCF 专属参赛凭证
+ 决赛答辩队伍成员可获合作企业优先面试机会

01 一等奖
2 名 · 共享一等奖奖金池

02 二等奖
3 名 · 共享二等奖奖金池

03 三等奖
6 名 · 共享三等奖奖金池

赛事日程

2026 年 10 月 — 11 月

10/10 – 11/02 01 **参赛者比赛准备**
可多次提交智能体打包镜像，平台自测并返回运行日志与性能指标。

10/24 – 10/25 02 **打榜平台开放测试（预计）**
平台预计在此期间开放测试。现开始进行参赛最终确认，以便分配大模型 API。请尽快加入比赛 QQ 群 1003959691，并在在线文档中完成确认。

本次测试模型，上下文最大 256k；每个队伍约 1 亿 Token 额度

11/02 – 11/08 03 **参赛者初赛打榜**
完成打分后公开成绩排行榜，组委会将依照成绩排名择出优秀队伍进入答辩阶段。

打榜材料

下载完整文档

使用 agenthack/base:1.1.1 开发、测试并提交自己的 Agent 镜像。
每个赛题的任务、输入输出格式和评分说明，请到对应赛题页面查看。
平台运行时优先读取注入的模型地址、Key 和模型名，避免写死配置影响评测。
查看各赛题文档

▼ 参赛选手开发指南

> 基础镜像：`agenthack/base:1.1.1`
> 本指南介绍如何使用赛事提供的基础镜像开发 Agent、进行本地测试，以及打包提交参赛作品。

▼ 一、开发与提交流程

赛事提供了预装运行环境和 AgentHack SDK 的 Docker 基础镜像。各参赛队伍可以在此基础上编写 Agent，设计自己的算法和工具，并在本地完成测试。

开发流程： 导入基础镜像 → 编写 Agent → 本地测试 → 导出并提交镜像

开始前，请留意以下约定：

- **模型调用：** 组织方会为每支队伍单独分配 API Key，并提供模型接口地址和模型名称。Agent 通过比赛指定的 LLM 网关调用模型。
- **自定义工具：** 支持自行编写 Python 工具、文档解析程序、离线检索模块等。自定义工具函数必须使用 SDK 的 `@tool` 装饰器，以记录实际执行的工具名称和调用信息。正式评测时，自定义Tool不允许访问互联网或第三方 API；模型调用统一走赛事指定网关。
- **运行数据：** 评测数据由平台提供。Agent可以读取允许访问的本地数据，也可以使用预先打包在镜像中的离线资源。
- **作品提交：** 最终提交的是包含Agent代码及所需依赖的**Docker镜像文件**，不是官方基础镜像或单独的Python源文件。

▼ 二、准备开发环境

1. 安装 Docker

请先安装并启动 Docker。Linux 可以使用 Docker Engine；Windows 推荐 Docker Desktop + WSL 2；macOS 可以使用 Docker Desktop。

下面的命令以 **Bash 终端**为例，适用于 Linux、macOS 或 Windows WSL 2。

2. 导入赛事基础镜像

将组织方提供的 `agenthack-base-1.1.1.tar` 下载到本地，在该文件所在目录执行：

```
docker load -i agenthack-base-1.1.1.tar
```

然后查看镜像：

```
docker images agenthack/base
```

如果列表中出现 `agenthack/base:1.1.1`，说明已成功导入。

> 💡 官方基础镜像提供运行环境和 SDK，后续还需要加入自己的 Agent 代码并重新构建镜像。

▼ 三、编写 Agent

1. 创建项目

新建文件夹 `my-agent`，项目最初可以采用下面的结构：

```
my-agent/
├── agent.py       # Agent 代码
└── Dockerfile     # 镜像构建文件
```

2. 编写 `agent.py`

下面是一个简单的 Agent 示例：先使用自定义 Tool 处理输入，再调用比赛提供的 LLM 生成回答。

```
import os
from openai import AsyncOpenAI
from agenthack import BaseAgent, AgentRuntime, tool
```

```
# 自定义工具：清理文本中的多余空格
@tool(name="clean_text")
def clean_text(text: str) -> str:
    return " ".join(text.split())


class MyAgent(BaseAgent):
    async def solve(self, task_input: dict) -> dict:
        question = str(task_input.get("question", ""))
        context = str(task_input.get("context", ""))

        # 处理输入
        question = clean_text(question)

        # 调用比赛指定的 LLM
        async with AsyncOpenAI(
            base_url=os.environ["PLATFORM_LLM_URL"],
            api_key=os.environ["PLATFORM_API_KEY"],
            timeout=20,
            max_retries=0,
        ) as client:
            response = await client.chat.completions.create(
                model=os.environ["AGENT_MODEL_NAME"],
                messages=[{
                    "role": "user",
                    "content": f"材料：{context}\n问题：{question}",
                }],
                max_tokens=256,
            )

        answer = response.choices[0].message.content or ""
        return {"answer": answer}


if __name__ == "__main__":
    AgentRuntime(MyAgent()).run()
```

Agent 通过 `solve(self, task_input)` 接收题目，在函数内部完成处理，并返回包含 `answer` 的字典，例如 `{"answer": "回答内容"}`。示例中的 `question`、`context` 用于演示。不同赛题的实际输入字段，请以对应赛题规则为准。

选手只需实现 `solve()` 中的业务逻辑，并为自定义工具函数添加 `@tool`。 程序最后一行的 `AgentRuntime(MyAgent()).run()` 负责启动服务，并提供两个接口：

- `GET /health`：检查 Agent 服务是否正常运行。

- `POST /solve`：接收任务并返回执行结果。

3. 添加自定义 Tool

自定义 Tool 可以是 Python 函数，也可以调用其他依赖库实现 PDF 解析、文本处理、离线文献检索或数学计算。凡作为工具执行的自定义函数，必须使用 SDK 的 `@tool` 装饰器。

例如，在 `agent.py` 中定义并调用工具：

```
from agenthack import tool

@tool(name="calculator")
def calculator(a: float, b: float) -> float:
    return a + b

# 在 MyAgent.solve() 内按需调用：
# result = calculator(2, 3)
```

关于网络访问： 正式评测环境中，自定义 Tool 不能访问网页、搜索引擎、在线数据库或第三方 API。工具可以处理容器内文件和平台提供的数据。LLM 调用则通过赛事指定的模型网关完成，不允许连接其他外部模型服务。

4. 编写 `Dockerfile`

在项目目录中新建 `Dockerfile`（文件名不带扩展名）：

```
FROM agenthack/base:1.1.1
WORKDIR /app
COPY --chown=agent:agent agent.py /app/agent.py
LABEL org.agenthack.team_id="team-001"
LABEL org.agenthack.version="1.0.0"
USER agent
EXPOSE 8080
CMD ["python", "agent.py"]
```

此文件会将 `agent.py` 加入官方基础镜像，并指定容器启动后运行该程序。

如需安装其他 Python 库，可在项目中添加 `requirements.txt`，并在 Dockerfile 的 `COPY agent.py` 之前加入：

```
USER root
COPY requirements.txt /app/requirements.txt
```

```
RUN pip install --no-cache-dir -r /app/requirements.txt
USER agent
```

请在构建镜像时安装好所需依赖。正式评测时容器无法联网下载依赖，也无法访问选手电脑上的文件。请保留官方 AgentHack SDK，并使用非 root 用户启动容器。任务和支持的模型/网络调用由 SDK 自动采集；自定义工具函数必须使用 `@tool`。

▼ 四、本地运行与测试

以下命令在 `my-agent` 项目目录执行。

1. 构建镜像

```
docker build --platform linux/amd64 -t my-agent:1.0 .
```

构建完成后，本地会生成一个名为 `my-agent:1.0` 的 Docker 镜像。

2. 配置模型 API

组织方将**手动向每支参赛队伍发放专用API Key**，同时提供模型接口地址和模型名称。请妥善保管本队的 Key，不要上传至公开代码仓库。

在 Bash 终端中配置环境变量（填写队伍收到的模型接口地址和模型名称）：

```
export PLATFORM_LLM_URL="组织方提供的模型接口地址"
export AGENT_MODEL_NAME="组织方提供的模型名称"
read -r -s -p "请输入本队 API Key：" PLATFORM_API_KEY; echo
export PLATFORM_API_KEY
```

打榜前，选手使用自己的API Key进行测试。

正式打榜时，选手需将组织方提供的API Key、模型地址写入镜像。

3. 启动容器

```
docker run -d --name my-agent-test \
  -p 127.0.0.1:8080:8080 \
  -e PLATFORM_LLM_URL -e PLATFORM_API_KEY -e AGENT_MODEL_NAME \
  -e AGENTHACK_TELEMETRY_REQUIRED=false \
  my-agent:1.0
```

这条命令会在本机启动 Agent，并将服务映射到 127.0.0.1:8080。

4. 测试接口

先检查服务状态：

```
curl http://127.0.0.1:8080/health
```

响应中出现 "status": "healthy"，表示服务已正常启动。

接着发送一条测试任务：

```
curl http://127.0.0.1:8080/solve \
  -H 'Content-Type: application/json' \
  -d '{"task_id":"test-001","input":{"question":"请介绍一下你自己"}}'
```

正常执行时，响应中会包含类似下面的内容：

```
{
  "task_id": "test-001",
  "status": "success",
  "output": {"answer": "这里是 Agent 返回的答案"}
}
```

示例只展示了主要响应字段。success 表示这次任务成功执行，并不代表回答一定正确。

若请求失败，可以查看容器日志：

```
docker logs my-agent-test
```

修改代码后，需要重新构建镜像并启动测试容器。原测试容器可通过以下命令删除：

```
docker rm -f my-agent-test
```

测试本地 PDF 或其他文件： 可以在项目中建立 test-data 目录，并在 docker run 命令中增加以下参数：

```
-v "$PWD/test-data:/data:ro" -e DATA_PATH=/data
```

容器内可以通过 /data/文件名 读取这些文件。正式评测时，比赛数据同样以只读方式提供；不同赛题的数据组织方式，请以对应赛题规则为准。

▼ 五、导出与提交镜像

完成本地测试后，导出参赛镜像：

```
docker save my-agent:1.0 -o my-agent.tar
```

完成本地测试后，在赛事平台对应赛题的提交入口上传 my-agent.tar。评测将使用你上传的镜像运行 Agent，并生成评测结果。

请注意区分：agenthack-base-1.1.1.tar 是组织方提供的基础镜像，my-agent.tar 才是包含参赛作品的镜像文件。

提交前检查

- [ ] 镜像基于 agenthack/base:1.1.1，架构为 linux/amd64。
- [ ] 容器启动后能够正常提供 /health 和 /solve 接口，服务监听容器内 8080 端口。
- [ ] solve() 的返回结果包含 answer。
- [ ] Agent 通过赛事指定网关调用 LLM，自定义 Tool 不访问外部网络。
- [ ] 实际执行的自定义工具函数均使用 SDK 的 @tool 装饰器。
- [ ] 镜像内包含运行所需的代码和依赖，不依赖本机文件或启动后的在线下载。
- [ ] API Key写入镜像。
- [ ] 保留官方 AgentHack SDK；任务和支持的模型调用自动采集，工具执行通过 @tool 记录。

▼ 常见问题

可以使用自己的 Agent 算法和工具吗?
可以。示例主要展示了任务输入、工具执行、LLM 调用和结果返回的基本写法。各队可以自行设计 Agent 的推理流程、提示词以及工具；

可以使用 LangChain 等 Agent 框架吗?
可以。请将使用这些框架编写的逻辑接入 BaseAgent.solve()，保留 AgentRuntime 提供的接口，并确保模型调用通过赛事指定网关完成。框架实际执行的自定义工具须经由带 @tool 的函数。SDK 自动适配预装的 OpenAI Chat Completions、HTTPX 和 Requests；其他模型客户端可以使用，但其模型 usage 可能显示为未观测。

可以通过 Tool 联网检索论文吗?

不可以。正式评测环境禁止自定义 Tool 访问互联网。可以使用平台提供的数据，或事先准备在镜像中的离线资源开展检索和处理。

可以调用其他大模型 API 吗?

正式评测只允许通过比赛指定的 LLM 网关使用允许的模型。各队收到的 API Key 对应赛事提供的模型接口。

需要自己搭建 HTTP 服务吗?

不需要另外搭建。示例中的 `AgentRuntime` 会启动服务并提供 `/health` 和 `/solve` 接口。

程序运行失败怎么排查?

可以先运行 `docker logs my-agent-test` 查看日志，再检查模型地址、模型名称、API Key、Python 依赖和任务输入字段。

修改代码后是否需要重新导出镜像?

需要。重新构建并完成本地测试后，再执行 `docker save`。平台评测的是最后上传的镜像版本。

主办单位：中国计算机学会（CCF） 承办单位：CCF网络与数据通信专委会、重庆大学

合作单位：华为技术有限公司

© 2026 全国高校 Agent 网络创新挑战赛 帮助中心 赛题 排行榜 Agent Native · Network Future

鄂ICP备2026056883号